GROUNDED DI LLC

Databricks Control Audit Record Book

Local control evidence and candidate repairs with a record of the harness itself

UNITY GATEWAY | SQL CONNECTOR | DI²

Pinned sources, preserved corrections and reproducible claims

27 October 2026
Independent public research edition

Independent research by Grounded DI LLC.
Not sponsored, endorsed or certified by Databricks.

RECORD NOTE

How to read this book

Grounded DI LLC examined a sequence of local Databricks client controls, retained the results, and later translated selected observations into registered tests. This book follows that sequence. It explains what the tests asked, what their recorded outputs establish, which controls held, and why several plausible concerns remain unresolved.

> The central result is bounded. Thirteen registered Databricks property and control tests produced six failing assertions across four issue groups and seven passing controls on the original local code. The same thirteen tests passed with separate candidate files. The DI² runner reported those results; its Gate did not enforce Databricks behavior.

The sources are five unique October 27 packages, including their nested earlier audits and snapshots. Six outer ZIP copies were supplied because the Boundary Audit appeared twice with identical bytes. Later explicit corrections qualify earlier claims without replacing the earlier records. In particular, the SQL retry observation reproduces a preexisting public report, and the native Claude parser was never executed.

For the original engineering edition, the supplied bytes were inventoried and hashed, duplicate evidence was compared, and historical outputs were reviewed. This public edition is a targeted disclosure edit of that book: it did not rerun the packaged tests, contact services, modify repositories, deploy candidates, or contact Databricks.

Two reading paths

Read pages 4 through 25 for the engineering story. Exhibits transcribe or explain historical evidence, not new test executions. Case identifiers connect findings to a privately retained claim-to-evidence record. The underlying ZIPs, member-level hashes and complete replay inputs are not public attachments; this PDF alone is not an independent replay kit.

The two supplied Microsoft screenshots informed only the visual design. No Microsoft metrics entered this book, and no unseen pages of that reference were reviewed. The earlier withdrawn PDF was not used. The separate Fallen Warriors Draft 3 PDF inside the supplied harness package was reviewed as source material for its stated method.

Record basis C01-C03 | Historical audit inventory, source-lineage evidence and book methodology are retained privately; no target tests were rerun for this edition.

CONTENTS

The record in sequence

Section	Page
Executive findings	4
Source lineage and preservation	5
Pinned targets and evidence language	6
Unity Gateway parser investigation	7–8
Cursor configuration preservation	9–10
SQL retry and the failed first repair	11–12
Five control review and environment correction	13
Decimal inference	14
Boolean conversion	15
Manifest copying	16
Host classification	17
Development token persistence	18
Fallen Warriors property translation	19–20
Audit the auditor	21–23
Lessons and the next evidence	24–25
Exhibit and count reconciliation	26–27
Reproduction references	28
Source identities and evidence boundaries	29
Public claim ceiling and release scope	30

The sequence is chronological within each investigation. Findings remain attached to their original target, inputs and method. A later candidate result does not erase a failed earlier candidate; a stronger recheck does not turn an earlier incomplete probe into a full execution.

Navigation note | Page numbers refer to this fixed-layout public edition. The original engineering book and all complete audit records remain privately retained and unchanged.

EXECUTIVE FINDINGS

What the evidence supports

13	6	4	7
registered Databricks tests	original failing assertions	issue groups	passing controls

The clearest implementation failures are small and traceable. Decimal inference used the text of scientific notation to size a SQL decimal cast. A filter helper returned the same manifest object despite promising a copy. A retry policy requested a thirty-second wait for a two-second header under the selected settings, and could reject a retry whose header value fit within the remaining budget. These conclusions concern local methods, not a live Databricks service.

Cursor presented a different kind of result. Both direct and batched registration replaced a malformed synthetic configuration. That can destroy recoverable bytes, but the reader intentionally treats malformed JSON as empty and does not expressly promise recovery. The later preservation tests express a proposed safeguard, not an established authorization rule.

Boundary	Disposition in this record
Decimal inference and manifest copy	FAIL against scoped representation or copy behavior
Retry header timing	Local discrepancy; preexisting issue #860
Cursor malformed configuration	PARTIAL preservation concern; no bypass shown
Boolean strings and host with port	PARTIAL; invalid-input or port contract unresolved
Development token host separation	PASS for the synthetic tested inputs
Native Claude duplicate options	UNRESOLVED; parser not executed

Separate candidates went green on their stated tests. The retry repair first went red on two upstream controls, then was narrowed. Grounded DI also found a defect in its lean harness package: the manifest schema did not match the verifier. Repairing that schema exposed a child-process cache problem, which the record investigated rather than concealing.

> What the exercise taught Grounded DI: stable case identifiers, explicit expected properties, counterevidence and environment receipts matter as much as a green summary. Repeatability alone cannot decide whether the property is the right one.

Evidence C04-C09 | Original case results, candidate comparisons and issue-group reconciliation are retained in private evidence records.

SOURCE LINEAGE

Five packages and their predecessors

The outer files are containers, not independent corroborations. Fallen Warriors includes byte-identical copies of the Boundary, Retry and Five Control packages. The Boundary package in turn carries the closure attempt, which carries the relayed-parser resolution, which carries the earlier continuation audit. Five Control contains five individual case archives. The source index retains each occurrence while the narrative counts a repeated observation only once.

EXHIBIT 01 ARCHIVE LINEAGE EDITORIAL DIAGRAM

```
Fallen Warriors (W)
  -> Boundary (B) -> Closure (C) -> Resolution (R) -> Continuation (U)
  -> SQL Retry (S)
  -> Five Control (F) -> D1, D2, D3, D4, D5
  -> original lean harness (L) and separate manifest candidate (LC)
Public Review (P) -> selected copies of Boundary evidence
Boundary second outer copy -> identical bytes, not another audit
```

Editorial lineage diagram. Exact nested archive paths and hashes are retained privately and are not part of the public release.

The Public Review is a derived presentation of the MCP evidence. Seven retained plan, test and output files byte-match the Boundary record. It omits the target source, nested history and candidate implementation. Its candidate result cannot be replayed from that small package alone; it adds no independent test population.

Preserving evidence without multiplying it

The original inventory used SHA-256 to establish equality among supplied bytes. It distinguished archival repetitions from independent evidence. Those historical comparisons do not authenticate authorship, establish an independent witness, or prove test correctness. Exact private file-level indexes are retained by Grounded DI LLC and are not released with this PDF.

The privately retained supporting record contains indexes and source references, not another copy of every historical archive. The original ZIPs remain unchanged and private. A file-level index covers nested archive members, including opaque assets not relied on as substantive evidence. Historical inventory counts were 39 archive occurrences, 17 unique containers, and 1,262 distinct non-container file contents. These are storage counts, not findings.

> The full historical harness and its controlled-replay input are named and hashed in the supplied receipts but are not included as full archives. This book inspected those receipts, not the absent archives. The smaller lean package is not a substitute for them.

Evidence C02-C03, C10 | Private archive inventory and byte-identity comparisons; duplicate archives are not independent corroboration.

METHOD

Pinned targets and evidence language

The audits pinned the relevant upstream implementations before interpreting behavior. Unity Gateway is pinned to commit b8c80ce4cbce09ea0b11a162136c312917a5853e. SQL Connector is pinned to e7a02cdec620d77502f359138ac1bd693df42c64, with distribution version 4.8.0 recorded for the installed-module work. These are historical targets, not claims about current releases.

The retry audit used exact selected files inside a minimal package shim. Five Control used an installed wheel and byte-matched the five target modules to pinned snapshots. Fallen Warriors checked the selected original SQL and Cursor files again. None of these checks binds every imported dependency or a hosted service to the same source revision.

Term	Meaning here
Observed behavior	A preserved output or state transition under declared inputs.
FAIL	An assertion or scoped contract expectation did not hold; the unit is stated.
PARTIAL	Behavior is shown, but contract, reachability or consequence is incomplete.
PASS control	The named expectation held for the tested inputs, not universally.
UNRESOLVED	The deciding evidence was not obtained.
Candidate	Separate local experimental code or packaging, not an adopted fix.
Setup error	The intended target test did not execute correctly; retained, not a finding.

A green test can preserve a red observation

Some early tests were written to confirm an observed overwrite. Those tests passed when the overwrite occurred. Later sidecars asserted that the damaged bytes should be preserved; those assertions failed on the same original behavior. The expectations changed, not the underlying evidence. Likewise, a harness replay can report PASS because it correctly reproduced an expected red/green pattern.

Fresh-process repeats establish consistency of the chosen cases, not additional distinct cases. Selected upstream suites provide useful counterevidence but are not pooled into a product-wide coverage score. Python socket hooks are test aids, not operating-system network isolation. Mocks replace services, authentication, clocks or process boundaries where the record says they do.

Evidence C11-C13 | Pinned upstream commit identities (p. 29), original module and environment records; only selected sources were bound.

CHAPTER 01 UNITY GATEWAY

Following an argument to the boundary

Unity Gateway’s relayed Claude launch is intended to exclude general user settings while supplying its own registrations. The source explains a practical reason: a stale user-scope apiKeyHelper should not shadow subscription OAuth. The builder adds --setting-sources project,local before caller arguments. The question was whether a caller could supply a conflicting --setting-sources user and change the downstream selection.

An argument list is an instruction to another program. Two conflicting options show what the sender passed; they do not show what the receiver ultimately loaded. A parser might choose the first, choose the last, combine them, or reject them. The investigation reached several sender-side boundaries without obtaining the native receiver’s answer.

The evidence grew in stages

The oldest retained audit could not collect tests with pytest or import the full module because dependencies were missing. It extracted three unchanged function bodies with Python’s syntax tree tools and supplied stubs. Eight assertions passed per process, but this did not demonstrate an integrated launch or real persisted state. The failed setup receipts remain in the record.

The continuation obtained compatible dependencies, though not the exact upstream lockfile, and imported the real managed_config module. A selected upstream suite recorded 174 passes. Eight frozen real-module cases passed in each of two processes, including refusal of a workspace mismatch before cache or token access and preservation of a persisted fallback and timestamp during mocked authentication failure. An expansion recorded 36 upstream passes on argument construction, ownership, fallback and picker cases.

EXHIBIT 02 RECORDED ARGUMENT SCOPE FIELDS

```
control_scopes: ["project,local"]
edge_scopes:    ["project,local", "user"]
downstream_cli_executed: false
```

Selected fields from two retained argument probes; rounds agree. This is a builder observation, not native parser output.

A separate probe exercised the real relayed launcher with login, proxy, thread and subprocess work mocked. It passed the conflicting list to a fake subprocess. These are useful reachability observations, but their mocked seams must remain visible.

Evidence C14-C16 | Selected launcher and argument probes, upstream test receipts and pinned Unity source; native parser not executed.

CHAPTER 01 CONTINUED

Why the parser question stayed open

The resolution stage moved closer to a normal command. A seven-form matrix used the actual Typer entrypoint with _launch_tool replaced, then called the real builder in the observer. Another three-case test ran both the real entrypoint and _launch_tool under an upstream helper’s relayed-provider setup. Space and equals forms reached a mocked launch_agent unchanged. Each of two fresh pytest processes recorded three passes.

EXHIBIT 03 OBSERVATION BOUNDARIES EDITORIAL DIAGRAM

```
Caller options -> real Typer -> real _launch_tool
                                -> MOCKED launch_agent
Caller options -> real relayed launcher -> real argv builder
                                -> MOCKED subprocess
Native Claude parser            -> NOT EXECUTED
```

The rows summarize separate mocked probes, not one uninterrupted end-to-end trace. Exact private test paths are withheld.

A conditional external-web-search guard did reject both setting-sources forms when a verified generated registration made that path applicable. The first probe’s stub mistakenly returned the same configuration for every local and project path, creating a fictitious overlap. The corrected stub restricted its answer to the intended path. Two corrected runs then showed the expected rejection. The passing control deliberately stopped at a mocked unavailable managed scope, so it did not prove a complete launch.

The closure attempt preserved another qualification. In explicit external mode, a simulated lower-layer scope conflict propagated as an error. In automatic mode, the wrapper warned, selected provider ucode and returned the caller’s original options. The lower-layer guard therefore was not evidence of a universal launch rejection.

> Disposition: local conflicting arguments and propagation to mocked boundaries are confirmed. Native parser precedence, actual settings selection and any security consequence remain unresolved.

Claude Code 2.1.306 was the intended downstream research version. A retained npm wrapper did not provide an installed native parser. The record could not establish the required no-network, no-side-effect native-agent isolation, so it did not launch the binary. The next decisive evidence would be an isolated, version-pinned parser trace for both orderings and syntactic forms, followed by an observation of settings selection. No parser repair was implemented here.

Evidence C17-C19 | Parser-reachability probes, mocked process boundaries and corrected guard observations; private records retained.

CHAPTER 02 CURSOR MCP

Preserving a damaged configuration

Cursor’s MCP registrations live in a shared JSON document. The pinned writer promises to merge entries while preserving unrelated content. The audit asked a recovery question: what happens if that document is already malformed? It redirected the real writer to a disposable file containing the synthetic bytes shown below. No actual Cursor application, complete configure command or MCP service was exercised.

EXHIBIT 04 THE MALFORMED INPUT

```
before = b'{"mcpServers":'
operation = write_mcp_server_config("svc", ["ug", "proxy"])
observed = new JSON containing svc replaced the original bytes
return value = False
```

Synthetic input and return value from a retained local probe; direct and batched registration showed the stated transition.

The mechanism is direct. read_json_safe converts malformed input into an empty object. The writer adds its new registration to that object, and write_json_file writes the resulting JSON. The old bytes no longer remain in that file. A person might have repaired some damaged contents manually, so overwriting can matter even when parsing failed.

The limit is equally important. The source deliberately treats malformed JSON as empty, and its general preservation language does not explicitly promise damaged-file recovery. The evidence supports a local preservation concern and a proposed preserve-and-deny policy. It does not establish a customer incident, unauthorized access, a hosted authorization bypass, or a definite violation of a malformed-input contract.

The return value is not an authorization decision

The direct writer’s False means that the parsed map did not already contain the named server. It does not mean a permission check failed. The batch path likewise works with registrations and removals, not a demonstrated general identity boundary. The tests do not establish caller authorization, ownership policy outside the tested helper, filesystem permission behavior or concurrent-write behavior.

> The original seven custom tests passed because they asserted the recorded observations and controls. Later sidecars made preservation the expected property and therefore went red on the original. Those results are consistent once the expectation is stated.

Evidence C20-C21 | Local malformed-JSON writer probe, selected Cursor source and original/candidate behavior records.

CHAPTER 02 CONTINUED

Controls that held and a narrower writer

The valid-file controls matter. An unrelated mine registration survived. A same-name svc registration was replaced and the return value reported replacement, as the function explicitly permits. That collision is not a second unauthorized-overwrite finding. A batch add/remove overlap produced the recorded remove-then-add behavior while retaining mine; the audit did not invent a stronger priority contract.

Claude's separate web_search ownership logic also held in its tested cases. A custom entry and unreadable configuration remained byte-identical, with no mocked remove/add calls. A matching generated entry permitted the mocked remove/add sequence. Because the mutation functions were mocked, the generated case does not show a real Claude rewrite. It shows that the ownership decision reached the expected seam.

Recorded MCP population	Count and meaning
Selected upstream suite	29 passed
Original custom cases	7 passed per fresh process: 4 Cursor + 3 Claude guard cases
Separate candidate cases	10 passed: 9 invalid-input combinations + 1 composite valid-controls case

The candidate changes recovery policy

The separate candidate distinguishes a missing file from malformed, non-object or structurally invalid content. It rejects the latter before writing. The nine invalid-input cases combine three payloads with direct, batch and removal operations. The tenth case exercises missing-file creation, intentional replacement, addition, deletion, metadata and unrelated-entry preservation.

This is stronger evidence for a proposed policy than merely showing that one malformed fixture is rejected. It is still a local candidate. The recorded matrix does not establish permission-error behavior, atomic writes, race resistance, other clients or whole-command compatibility. The candidate was not merged or deployed, and the original source was not changed.

> A useful repair preserves the valid cases as well as refusing the damaged ones. Here the tests support a specific read-before-write safeguard; they do not convert a shared-file helper into an authorization system.

The small Public Review package repeats selected Boundary outputs and adds explanatory material. Its name is a historical package title, not an instruction to publish this book. It is retained as lineage, with no independent count added.

Evidence C22-C24 | Valid-file controls, selected upstream tests and separate candidate results; private case receipts retained.

CHAPTER 03 SQL RETRY

When two seconds becomes thirty

Retry-After tells a client how long to wait before trying again. The selected SQL connector method says it sleeps for that header duration, yet applies max(proposed_wait, delay_max). With delay_max set to thirty, the expression acts as a floor. A two-second header therefore became a thirty-second requested wait in the recorded local probe.

The audit copied the pinned retry and exception files into a minimal package shim. A synthetic response supplied the header; time and sleep were mocked. The clock and start time were fixed at 100. There was no actual delay, HTTP request, credential or database action. The exhibit records requested sleeps, not elapsed wall-clock measurements.

EXHIBIT 05 RECORDED RETRY OBSERVATIONS

Header	Total budget	Requested sleep	Result
2 seconds	900 seconds	30 seconds	Returned True
30 seconds	900 seconds	30 seconds	Control passed
60 seconds	900 seconds	60 seconds	Returned True
2 seconds	10 seconds	None	MaxRetryDurationError

The budget case explains the practical concern. A two-second header would fit inside ten seconds, but the proposed thirty-second wait is rejected before sleep. This establishes a local retry-availability discrepancy under the selected inputs. It does not establish an outage or how a real server and connector transport would behave.

The sixty-second row needs a different interpretation. A server’s longer delay may be a minimum the client should respect. Exceeding a parameter called a maximum is not enough, by itself, to prove that honoring the server is wrong. The stronger finding is the inflation of a small positive header. A second probe through inherited urllib3 Retry.sleep confirmed dispatch to this override under the same mocks, not a full request path.

Status decisions supplied counterevidence

Direct method controls declined a synthetic POST ExecuteStatement 500 without an override, accepted 429, accepted forced 500 under the documented override, and in server-directed mode required a header for 429. These controls prevent a timing discrepancy from becoming a blanket claim that retry safety failed.

Evidence C25-C27 | Pinned SQL retry method, synthetic header and clock probes; public issue #860 (p. 29).

CHAPTER 03 CONTINUED

The first repair broke the controls

Twelve pinned upstream retry tests passed on the original. Two expressly expected the thirty-second floor when no Retry-After header was present. Those tests exposed a compatibility assumption that a narrow counterexample alone could miss.

Candidate v1 removed the floor in both the header and no-header branches. Its five new cases passed, but two upstream controls failed. The first expected sleep(30) and observed sleep(1). The multiple-retry test expected thirty seconds each time and observed 1, 2 and 4. The retained result was fifteen passed and two failed, not a clean repair.

EXHIBIT 06 THE REGRESSION WAS RETAINED

```
Candidate v1
  Expected: sleep(30)
  Actual:   sleep(1)
  2 failed, 15 passed

Refined candidate v2
  17 passed
```

Recorded expected/actual values from retained candidate comparisons. Both belong to the 12 upstream plus 5 new case population.

Candidate v2 changed only the positive-header path. It retained the no-header floor, respected long server delays and kept the total-duration guard. All seventeen cases passed: twelve upstream and five new cases, including the two-second header under a ten-second budget and rejection of a sixty-second header under that budget. The candidate remains an unmerged local suggestion, not evidence of transport-wide compatibility.

A later correction changed the novelty claim

The original retry archive did not identify a prior issue. The later Five Control evidence index explicitly records public issue #860, opened on July 14, 2026, as having already reported the same max(wait, delay_max) behavior. The prior test reproduced that existing report. The book applies the correction while retaining the original archive and failed candidate unchanged. The date is attributed to the supplied correction, not a fresh web check.

> This sequence is the repair lesson: keep the counterexample, keep the old controls, and narrow the change until the evidence supports both. A later green result is more informative when the failed first attempt remains visible.

Evidence C28-C30 | Twelve upstream and five additional retry cases, including failed first candidate; public issue #860.

CHAPTER 04 FIVE CONTROLS

Different questions need different verdicts

The next package broadened the review across five SQL connector boundaries. It selected isolated local methods rather than live authentication or database operations. The outcomes were two FAIL, two PARTIAL and one PASS. That distribution is part of the story: a surprising output is not automatically a defect, and a control that holds deserves the same preservation as a failure.

Case	Scoped status	Upstream tests
Decimal cast inference	FAIL	56
SEA boolean conversion	PARTIAL	6
Filtered manifest copying	FAIL	2
OAuth host classification	PARTIAL	3 plus 8 subtests
Development token persistence	PASS	2

The upstream denominator is sixty-nine pytest tests. The eight endpoint subtests sit inside its three-test suite and are reported separately, not added to make seventy-seven. The probe outputs were recorded in two fresh processes and in a clean environment. Static comparison for this book found the corresponding JSON observations byte-identical; that comparison is not another execution.

The environment correction strengthened the record

The initial environment contained two python-dateutil metadata records, 2.9.0.post0 and 3.0.0, while importing 2.9.0.post0 code. The clean environment had one metadata record, 2.9.0.post0, and reproduced the observations and suites. The initial endpoint output was genuinely empty despite a recorded zero exit. A separate recheck captured three passes and eight subtests, and the clean run captured them again. Neither an empty log nor an exit code alone substitutes for visible results.

The five installed target modules byte-matched the pinned snapshots. The package does not bind every transitive import, bundle all wheels or guarantee offline supply-chain reproduction. Its environment notes describe cleared credentials during the recorded runs, but the supplied replay scripts do not themselves construct a fully cleared env -i environment. A socket guard is not a security sandbox.

> Read the next five cases as separate contracts: representation, conversion, copying, hostname classification and host-keyed persistence. Their evidence cannot be collapsed into a single product score.

Evidence C31-C33 | Five-control case records and clean-environment comparison; tests were local, not hosted.

CHAPTER 04 DECIMAL

The spelling of a number is not its size

A SQL DECIMAL(p,s) has p total digits and s fractional digits. The parameter class promises an inferred cast able to contain the input, using the smallest appropriate representation. Scientific notation makes a useful boundary test: the short string 1E+20 denotes a number that needs twenty-one integer digits, while 1E-7 requires seven fractional places.

The probe built real DecimalParameter objects and read the generated Thrift type and value. An independent Python decimal-arithmetic check examined representability. It did not submit the parameter to a server, so it cannot tell us whether a real query would reject, round or otherwise handle the mismatch.

EXHIBIT 07 ORIGINAL OUTPUT AND REGRESSION EXPECTATIONS

Input	Original type	Candidate expected type
1E-7	DECIMAL(4,0)	DECIMAL(7,7)
1E+20	DECIMAL(5,0)	DECIMAL(21,0)
-0.01	DECIMAL(4,2)	DECIMAL(2,2)
12 with options (5,0)	DECIMAL(2,0)	DECIMAL(5,0)
1234.56 control	DECIMAL(6,2)	DECIMAL(6,2)

The source inferred size from the string and split it at a literal decimal point. Scientific notation therefore became a character-count problem; a minus sign also enlarged the inferred type. Separately, a truthiness check treated an explicitly supplied zero option as absence. The API names its total-digit option scale and fractional-digit option precision; the record preserves that unusual naming rather than silently swapping it.

All fifty-six upstream parameter tests passed. Their decimal examples were ordinary positive non-scientific values. Seven new regression cases made the boundaries visible: the original failed four and passed fifty-nine overall. Two failures concerned representability, one concerned a nonminimal cast and one concerned the ignored explicit zero. They are not four independent vulnerabilities.

The separate candidate used Decimal.as_tuple() for finite inference and is not None for explicit options. It passed all sixty-three cases. This supports the tested repair without establishing every decimal, precision limit, nonfinite value or end-to-end cursor path. The absence of a server result remains a material limit.

Evidence C34-C36 | Pinned decimal parameter source and original/candidate 56+7 case population; private outputs retained.

CHAPTER 04 BOOLEAN CONVERSION

A clear output and an unclear invalid input policy

The SEA result converter maps recognized true strings to True and other strings to False. The audit asked what happens to values outside the familiar true/false pair. It called the converter directly with synthetic inputs, without a service response or database.

EXHIBIT 08 RECORDED CONVERSION OUTPUTS

```
boolean "true"     -> True
boolean "false"    -> False
boolean "unknown"  -> False
boolean "TRUE "    -> False
int "not_a_number" -> "not_a_number"  (str)
```

Recorded synthetic converter outputs; the invalid-integer branch logged a warning. Repeated and clean results matched.

The implementation lowercases the string and checks membership in a true-value list. It does not strip whitespace. Thus TRUE followed by a space is a whitespace case, not evidence that ordinary uppercase TRUE fails. An unknown string is outside the list and becomes False without raising an exception.

The invalid integer control followed a different path: conversion raised an exception, which the general handler logged before returning the original string. That contrast shows how the code behaves. It does not create an implicit promise that every invalid boolean must also raise or preserve the original input.

> Disposition: PARTIAL. The mapping is demonstrated, but the record identifies neither a required invalid-boolean policy nor evidence that the SEA service supplies these malformed values.

What held and what would settle it

All six upstream conversion tests passed, including valid true and false spellings. Multiple assertions inside the boolean test are not additional pytest tests. No candidate change was proposed, because choosing rejection, preservation or coercion would first require an input contract and caller context.

The next evidence would be a versioned description of allowed service values and validation at the caller boundary. If malformed external strings can reach this method, the required disposition should be explicit. Without that evidence, the useful engineering result is a documented conversion policy and an unresolved precondition, not a security claim.

For Grounded DI, this case checks the auditor’s restraint. A test can establish an output with high confidence while leaving the judgment about that output open. The confidence in observation and the confidence in defect classification are different questions.

Evidence C37-C38 | Converter source, synthetic input/output receipts and six selected upstream tests; contract unresolved.

CHAPTER 04 MANIFEST COPYING

Correct rows with shared metadata

A result manifest describes the result, including its total row count. A filtered view can contain two rows while the source still describes four. The helper’s stated task was to create a copy with an updated count. Instead, it assigned the original manifest to a new variable, changed the count and returned the same object.

The first probe called the helper directly and compared identity. A stronger recheck then built a real in-memory SeaResultSet and JSON queue, with synthetic TABLE and VIEW rows and mocked connection/backend. It filtered four rows down to two TABLE rows without replacing the result-set constructor. This strengthened the observation beyond an intercepted constructor call.

EXHIBIT 09 SELECTED REAL RESULT SET OUTPUT

```
"source_manifest_after": 2,
"filtered_manifest_after": 2,
"same_manifest": true,
"control_manifest_after": 4,
"filtered_rows": [["TABLE"], ["TABLE"]]
```

Selected fields from the in-memory recheck; the second fresh-process and clean runs retained the same observation.

The filtered rows were correct. The failure was metadata ownership: the new result and original result shared the manifest, and the original count changed from four to two. A separate unrelated manifest remained four. The source’s copy promise makes this a more definite contract failure than the boolean case.

Two upstream filter tests passed but did not assert original manifest identity or preservation. The focused regression failed on the original while those two controls passed. A separate shallow-copy candidate passed all three. It supports the tested count isolation, not deep independence for every nested field, concurrent use or every result path.

Keep the scope at the manifest

Filtering also consumes the source queue in the implementation. The finding is not that the whole source result must remain untouched, nor that every filtering operation is pure. It is the narrower inconsistency between a stated manifest copy and a shared mutable object. No server or Arrow path was exercised, and no caller’s observed downstream consequence was established.

> The lesson is about identity as well as value. Two objects can display correct filtered rows while still sharing metadata whose mutation reaches farther than the interface promises.

Evidence C39-C41 | Manifest filtering source, real in-memory result-set recheck, and isolated candidate receipts.

CHAPTER 04 HOST CLASSIFICATION

Recognizing a host without inventing a port contract

The OAuth endpoint helper classifies known cloud hostnames using suffixes. That classification influences which endpoint type is selected. The audit tested the classifier locally; it did not authenticate, route a real token or contact a domain.

EXHIBIT 10 HOST CLASSIFICATION CASES

Synthetic input	Recorded classification
foo.cloud.databricks.com	AWS and supported
HTTPS://Foo.Cloud.Databricks.Com/path	AWS and supported
foo.cloud.databricks.com:443	No classification
foo.cloud.databricks.com.attacker.invalid	No classification
example.invalid	No classification

The valid and lookalike controls are important. A hostname merely containing the known domain was not enough; the attacker.invalid suffix did not match. The known host with :443 also failed to match because the complete string no longer ended with the expected suffix.

That port observation is unambiguous, but its classification is not. The source calls the input a hostname, and the record did not identify an explicit promise to accept a port. A caller might be required to pass an already normalized hostname. Without that contract or a real caller path, the result remains PARTIAL rather than a demonstrated endpoint defect.

A missing capture was corrected

The original upstream endpoint capture contained zero bytes even though the recorded exit was zero. The audit did not rely on that blank as proof. A new-process recheck captured three passing tests and eight subtests; the clean environment reproduced those counts. The original empty capture remains indexed so the later evidence can be distinguished from it.

> No token-routing or authorization conclusion follows from this local classifier. The next evidence is the accepted input grammar and normalization performed by callers, especially whether a host-and-port string is supported.

This case illustrates two separate audit obligations. The target behavior needs a contract before it becomes a failure. The evidence capture itself needs an observable result before an exit status becomes persuasive. Both qualifications belong in the main narrative, not hidden in a footnote.

Evidence C42-C44 | Pinned host classification source, repeated synthetic cases and corrected upstream capture.

CHAPTER 04 TOKEN PERSISTENCE

The host separation control held

The development token-persistence case asked a simple question: could a lookup for one hostname return a synthetic token stored for another? The source includes a host-keyed in-memory cache and a file implementation explicitly labeled development-only that supports one hostname.

EXHIBIT 11 PERSISTENCE OBSERVATIONS SUMMARY

```
File cache
  matching host -> its synthetic token
  different host -> None
  malformed JSON -> None

Memory cache
  host a -> token a; host b -> token b
  unknown host -> None
```

Summary of repeated synthetic host-keyed persistence receipts. Tokens were test fixtures, not credentials.

The tested controls held in both fresh processes and the clean environment. Two pinned upstream persistence tests passed as well. The file implementation’s plaintext storage is visible in source, but it is explicitly development-only. The record does not relabel that documented design as a production security defect.

This is a scoped PASS. It establishes neither production credential protection nor operating-system access controls. It did not test real tokens, concurrent writes, permission modes, authentication or all malformed inputs. Those are different questions requiring their own evidence.

Why a passing case belongs in the book

The audit was not a search for a uniformly negative result. Host separation was a meaningful control and it survived the selected probes. Later Fallen Warriors tests reused a host-keyed cache control alongside the failing decimal, manifest and retry properties. That passing control constrains the claim: the sidecar scan did not demonstrate a general collapse of SQL connector behavior.

A careful audit record should retain that result even if it attracts less attention than a failure. Removing it would distort both the original scope and the later thirteen-case denominator. The same principle applies to valid Cursor entries, ordinary decimals and the retry header-thirty control.

> The five-control chapter ends with an intentional asymmetry: two definite local failures, two unresolved contract questions and one passing boundary. Those are five different evidentiary outcomes, not five defects.

Evidence C45-C46 | Development token-store source, synthetic host isolation tests and passing controls.

CHAPTER 05 FALLEN WARRIORS

Turning observations into registered properties

The Fallen Warriors draft argues that replay alone cannot establish semantic correctness: an incorrect decision can be repeated consistently. It asks for explicit requirements, failures that remain visible, and the same standard applied to Grounded DI itself. The later package put selected earlier observations into separately authored sidecar tests, each with a stable identifier and an expected property.

The nine SQL tests and four Cursor tests ran through the supplied DI² unittest runner. They were explicitly selected sidecars, not new default cases installed inside the original harness. The runner registered and reported them. Its Gate did not wrap the client, authorize its operations or stop Databricks behavior.

EXHIBIT 12 THE THIRTEEN CASES BY PURPOSE

Issue group or control set	Original	Candidate
Decimal representability	2 FAIL	2 PASS
Manifest copy	1 FAIL	1 PASS
Retry header	1 FAIL	1 PASS
Cursor damaged file preservation	2 FAIL	2 PASS
Seven ordinary and boundary controls	7 PASS	7 PASS

Six failing assertions therefore represent four issue groups, not six vulnerabilities and not thirteen bugs. Decimal contributes two values; Cursor contributes direct and batched paths. Boolean conversion, the port question and native parser precedence did not become new sidecar findings merely because they appeared elsewhere in the record.

The original SQL summary records four FAIL and five PASS across nine cases, with no load or harness errors. Cursor contributes two FAIL and two PASS. All thirteen passed against separate candidate files. Both modes were repeated in fresh processes, and registered IDs and result bytes matched across rounds.

> The registered property matters. Cursor’s preservation expectation is a proposed safeguard, while the manifest expectation follows an explicit copy promise. A common test runner does not make their contract strength identical.

Evidence C47-C50 | Original/candidate outcomes for 13 registered sidecar tests; full records privately retained.

CHAPTER 05 CONTINUED

What repeatability proves and what it does not

Registered suffix	Original	Group or purpose
DEC_SMALL / DEC_LARGE	FAIL / FAIL	Decimal representability
DEC_CONTROL	PASS	Ordinary decimal
MANIFEST_COPY / MANIFEST_CONTROL	FAIL / PASS	Copy property / correct filtered rows
RETRY_HEADER / RETRY_CONTROL	FAIL / PASS	Header 2 / header 30
HOST_CONTROL / CACHE_CONTROL	PASS / PASS	Suffix rejection / host-keyed cache
BAD_SINGLE / BAD_BATCH	FAIL / FAIL	Proposed malformed-file safeguard
CONTROL_SINGLE / CONTROL_BATCH	PASS / PASS	Valid unrelated Cursor entry

SQL suffixes above have the prefix test_fallen_sql.SQLWarriors.test_. Cursor suffixes have test_fallen_cursor.CursorWarriors.test_. The supporting map preserves full IDs. Every candidate status in this population is PASS in both recorded rounds.

The crosscheck confirms the same registered IDs on originals and candidates, and byte-identical registries and results between each mode’s rounds. Static comparison for this edition independently confirmed those equalities. It is useful protection against accidental case loss or different case membership. It is not proof that the assertions express the correct specification, that dependencies are fully pinned, or that a live service would behave the same way.

Separate candidate files preserved the comparison

The SQL overlay combined prior local decimal, manifest and refined retry candidates. Cursor used its separate candidate module. Selected original files were checked against pinned snapshots. The record did not modify the installed originals or merge upstream. Real generated parameter objects and in-memory results were used where specified; clocks, connections and external effects remained mocked or synthetic.

The overall portable replay later reports PASS when these expected original failures and candidate passes are reproduced. Reading that headline as “all original controls passed” would reverse its meaning. The supporting count table distinguishes case count, assertion result, replay expectation and byte-equality comparison.

> Repeatability is evidence about stability. Correctness still needs a justified property and an execution boundary that actually reaches the behavior being claimed.

Evidence C49-C52 | Stable case IDs, recorded repeat outcomes and candidate identity comparisons; no hosted execution.

CHAPTER 06 AUDIT THE AUDITOR

The full harness receipts and their denominator

Grounded DI's own harness was part of the investigation. The supplied record contains outputs of a fresh full native replay at that historical stage. It records two rounds with 105 core assertions and 135 unit assertions per round, plus integrity, mutation, crash and repeatability checks. Those results must be read according to what each suite expects.

99	6	105
implementation cases	deliberate bug fixtures	core assertions per round

The six deliberately defective core fixtures, M01 through M06, have property FAIL but assertion PASS because the harness correctly classifies them as reproduced bugs. They are not the six failing Databricks sidecar assertions. Adding or equating the two sets would erase the distinction between a deliberately broken fixture and an original client property that did not hold.

EXHIBIT 13 SELECTED HISTORICAL HARNESS RECEIPTS

```
Core population: 99 implementation passes; 6 deliberate bug fixtures.

Post-run check: 3,619 recorded file entries.
Observed: PASS; no mismatches or unexpected files.
Security limit: unsigned membership/byte comparison.
```

Editorial condensation of historical receipts; source JSON and native test logs are privately retained. Not a new execution.

The direct post-replay verifier checked 3,619 entries. That is a file-membership and byte count, not a test count or a signed authenticity guarantee. The 135-test unit suite is a separate population; its second round repeats it. Mutation, crash and real-clock checks have their own units and are not added into a headline total.

The source boundary remains explicit

The complete full harness archive and the approximately 73 MB controlled-replay archive are not bundled in Fallen Warriors. Their hashes and execution receipts are. This edition therefore reviews the receipts and can trace their case counts, but cannot claim to have inspected the missing full archives or independently reproduced their execution. A future full replay needs those separately retained originals.

The record also preserves an initial wrong-working-directory setup that wrote a plan and empty directories into an extracted working copy. That copy was excluded; the input archive remained unchanged and a fresh extraction was used. It is a harness workflow correction, not a Databricks defect.

Evidence C53-C56 | Historical harness and integrity receipts; complete original replay archives are not included in this PDF.

CHAPTER 06 CONTINUED

A manifest its verifier could not read

The lean package's verifier expected path-keyed verification records. Its manifest instead used a descriptive list of files. The unchanged verifier rejected that incompatible schema before checking normal file membership and bytes. Its recorded result was ERROR, exit 2, including an invalid-manifest-record message. This was a Grounded DI packaging failure, not a Databricks defect.

EXHIBIT 14 SCHEMA MISMATCH EDITORIAL SUMMARY

```
Supplied package: descriptive list of file records
Native verifier: path-keyed membership and byte records
Observed: ERROR before normal integrity verification
```

Interpretive schematic, not literal manifest JSON. The original schema and verifier output are preserved privately.

A separate list-aware check found that all forty-six listed payload hashes matched. That useful byte check is not a native verifier PASS. The original lean ZIP contained forty-nine members: forty-six listed payloads, the self-excluded manifest, a checksum file and an unlisted Python 3.13 bytecode file.

The candidate converted the list to the verifier's mapping, added the existing checksum file as a hashed entry and omitted that unlisted bytecode. It preserved all forty-six original listed payloads and the checksum byte-for-byte. The candidate had forty-seven listed entries plus its self-excluded manifest. Its native verifier passed forty-seven entries; deliberate runtime modification and an added unlisted file then produced the expected FAIL reports.

Registered lean property	Original	Candidate
Native accepts a consistent package	FAIL	PASS
Frozen 46 payloads remain intact	PASS	PASS
Unlisted file is named as FAIL	FAIL	PASS
Changed runtime is named as FAIL	FAIL	PASS

The three original failures have one cause: schema incompatibility prevents normal verifier operation. They are not three separate defects. The sidecar itself completed; its failures compare expected native statuses with the schema ERROR. A separate initial candidate invocation used the wrong working directory and never opened the verifier. That setup error is retained separately from the real schema failure.

Evidence C57-C60 | Original schema error, separate payload checks and isolated repair comparison; private tests retained.

CHAPTER 06 CONTINUED

A passing test changed the sealed extraction

After the manifest candidate passed, its quick-start tests exposed another boundary. Running the parent Python with -B did not stop a child process from creating a Python 3.12 cache in the extracted working copy. The verifier then correctly rejected that unexpected file. This new cache was different from the Python 3.13 bytecode already present in the supplied original lean ZIP.

The frozen recheck isolated one adapter regression in two fresh candidate copies. In both, the test assertion passed. Without PYTHONDONTWRITEBYTECODE, the child-created cache made the post-test check fail. With the environment variable set to 1, the setting reached the child and the post-check passed.

EXHIBIT 15 RECORDED CACHE RECHECK EXIT CODES

```
cache_without_env before 0
cache_without_env test   0
cache_without_env after  1
cache_with_env    before 0
cache_with_env    test   0
cache_with_env    after  0
```

Recorded exit-status matrix: zero means success; after=1 is the verifier rejecting an extra cache, not a failed assertion.

A later clean candidate passed the full eighty-seven-test quick start, and native forty-seven-entry checks passed before and after. This establishes the tested mitigation for that process arrangement, not protection from every possible child. The unchanged lean README lacks the environment setting; the corrected replay instructions include it.

Portability had its own failed attempt

The first portable replay resolved a virtual-environment Python symlink to its base interpreter. It lost the environment and failed dependency detection before running tests. The correction retained the absolute environment executable without resolving the symlink. The revised replay recorded eighteen PASS expectation/equality records: twelve mode-and-round executions plus six comparisons, not eighteen independent target properties.

The assembled pre-QA archive was also extracted and replayed historically. Adding the QA note and outputs changed the final archive hash without changing code, fixtures, candidates or inputs. The receipt distinguishes the tested pre-QA ZIP from the final ZIP; the book does not pretend their hashes are equal.

> The harness lesson is operational: a passing assertion can still leave an extra file, and a familiar interpreter path can still lose its dependencies. Reproducibility includes process inheritance, working directories and the exact artifact being verified.

Evidence C61-C64 | Child-process cache and portability corrections, with original failed attempts retained privately.

SYNTHESIS

What changed in Grounded DI’s method

The exercise made the distinction between observation and judgment concrete. A decimal cast unable to represent its input has a stated representation contract. A changed manifest has a copy promise. An unknown boolean coerced to false needs an invalid-input policy before it can be called a violation. A damaged Cursor file invites a stronger recovery safeguard, but the safeguard must not be attributed to upstream code as an existing authorization contract.

Write the property before scoring the result

Registered sidecars made the expected behavior explicit and preserved case membership between original and candidate. That strengthened comparison, but it did not settle whether every property was the right specification. The Fallen Warriors method’s warning about replay and semantic correctness applies to the auditor’s own assertions as much as to the target.

Use passing controls to constrain a repair

Retry v1 is the clearest example. It improved the new cases while breaking the no-header behavior that upstream tests expected. Keeping that failed attempt made the refined repair explainable. The same discipline preserved ordinary decimal values, correct filtered rows, valid Cursor entries and host-keyed tokens alongside failures.

Treat the harness as another tested implementation

The lean schema failure was in Grounded DI’s packaging, not Databricks. The cache recheck then showed why integrity and test success answer different questions. The candidate packaging did not become trustworthy merely because its bytes were hashed: the verifier needed a compatible schema, and the extraction needed to remain unchanged after its child processes ran.

Preserve corrections as evidence

The existing retry report changed the novelty claim. The blank endpoint capture required a new capture. A malformed fixture needed a corrected stub. A virtual-environment path needed a portability correction. Retaining those stages lets a skeptical reader see why confidence increased or a claim narrowed, rather than receiving only a polished final green line.

> The practical output is a more disciplined record: explicit properties, bounded conclusions, separate candidates, stable identifiers, preserved failures and a replay path that states its missing inputs. None of that requires a product-wide verdict or an invented certification.

Synthesis C65 | Interpretation from scoped source, test, counterevidence and self-audit records; no product-wide score.

LIMITS AND NEXT EVIDENCE

What would resolve the remaining questions

Open question	Decisive next evidence
Claude duplicate-option precedence	Safely isolated native 2.1.306 parser trace, both orderings/forms, plus actual settings selection.
Cursor recovery policy	Explicit malformed-file contract and whole-command compatibility review, including permission and concurrent-write cases.
Decimal server consequence	Authorized end-to-end parameter execution for scoped values; distinguish rejection, rounding and exact preservation.
Boolean invalid-input handling	Versioned service representation and caller validation contract.
Host with explicit port	Accepted input grammar and normalization at the actual caller.
Candidate completeness	Broader regression coverage and direct import provenance for candidate modules.
Full harness replay	Separately retained full archives and declared dependencies, not only supplied receipts.

These are proposed evidence requests, not work performed by this public edition. No candidate was deployed, and no hosted service, pull request, upstream repository, or Databricks organization was altered or contacted in preparing this release candidate. Underlying audit ZIPs and complete execution records remain private; publication or outreach is a separate owner decision.

Limits that remain even after local repetition

The packages do not establish a customer incident, hosted authorization bypass or product-wide safety rating. The SQL shim is not the full transport. Selected wheel modules match pinned sources, but every dependency is not bound. Python socket guards are not operating-system confinement. Candidate tests demonstrate selected behavior, not upstream adoption or production readiness.

Some candidate plans ask for module __file__ verification, but the corresponding Five Control replay and outputs do not visibly print that provenance. The separate-copy patch and PYTHONPATH mechanism is recorded; a complete import-path proof is not. The book therefore says recorded candidate results, rather than claiming more precise execution identity than the receipts support.

The historical harness receipts do not verify a particular model identity or reasoning configuration, establish independent certification, or support patent conclusions. An unsigned manifest cannot authenticate who authorized its contents. These limits concern the claims they qualify; they do not erase the narrower observed local behavior described in the book.

Evidence C66-C68 | Known gaps and unresolved execution boundaries from private historical records; no fresh product tests.

APPENDIX A EXHIBITS

A guide to the short evidence exhibits

Exhibit	Subject	Page
01	Archive lineage diagram	5
02	Recorded argv scope fields	7
03	Separate mocked observation boundaries	8
04	Malformed Cursor input and transition	9
05	Retry header and budget outcomes	11
06	Failed retry candidate and refinement	12
07	Decimal original and candidate types	14
08	Boolean and integer conversion outputs	15
09	Shared manifest and correct filtered rows	16
10	Host classification controls	17
11	Host-keyed token persistence	18
12	Thirteen sidecars by purpose	19
13	Full harness classifications and integrity	21
14	Lean manifest schema mismatch	22
15	Cache recheck exit codes	23

How the exhibits were prepared

Diagrams are explicitly labeled as editorial diagrams. Other exhibits transcribe selected source fields, summarize a recorded transition or compare original output with a candidate test expectation. Captions identify which kind of representation is used. Whitespace may be normalized for readability; the unabridged source remains in its original archive.

A privately held claim map retains exact source-member hashes and test identifiers where applicable. It is not supplied as a public attachment. The figures in this PDF are text-based editorial exhibits, not reconstructed application screenshots.

> A short exhibit is an entry point into the evidence, not a replacement for it. The supporting record preserves the path from the readable claim back to the source bytes.

Evidence guide | Exact source-member hashes and test IDs remain in the private claim map, not in this public edition.

APPENDIX B COUNTS

Reconcile the units before adding them

Population	Recorded count	Correct interpretation
Five Control upstream	69 tests; 8 endpoint subtests	56+6+2+3+2 tests. Subtests not added.
Decimal comparison	Original 4 FAIL / 59 PASS; candidate 63 PASS	Same 56 upstream + 7 regression cases.
Manifest comparison	Original 1 FAIL / 2 PASS; candidate 3 PASS	Same 2 upstream + 1 regression case.
Retry comparison	v1 2 FAIL / 15 PASS; v2 17 PASS	Same 12 upstream + 5 new cases.
MCP original and candidate	29 upstream; 7 custom; 10 candidate	Different scoped sets, not 46 findings.
Databricks sidecars	13 = 9 SQL + 4 Cursor	6 FAIL in 4 groups + 7 PASS controls.
Lean sidecars	4 = 3 FAIL + 1 PASS originally	One schema issue; candidate 4 PASS.
Full native core	105 per round	99 implementation + 6 deliberate bug fixtures.
Full native unit	135 per round	Separate suite; repeat round not new cases.
Full native integrity	3,619 entries	File checks, not tests or authenticity proof.
Lean candidate integrity	47 entries	46 preserved payloads + checksum file.
Lean quick start	87 tests	Subset population, not full 135 unit suite.
Portable replay	18 PASS records	12 mode/round executions + 6 comparisons.

The earlier parser lineage records 8 extracted-body assertions per process, then 174 selected upstream passes, 8 real-module cases per process, a 36-pass expansion, a seven-form entrypoint matrix and a later three-case full-entry test per process. These are stage-specific populations with possible overlap, not one additive independent suite.

Two fresh runs improve repeatability evidence but do not double unique tests or issue groups. The source inventory also distinguishes archive occurrences, unique archive bytes, member entries and distinct file blobs. None of those storage counts is a count of findings.

Evidence C04-C64 | Original denominators and replay populations were reconciled; repeats are not new cases.

APPENDIX C REPRODUCTION

Where a future authorized replay would begin

Historical replay instructions and underlying tests remain private. No command was run in producing this public edition. Authorized replay would require retained sources, controlled dependencies, isolation and separate original archives. This PDF reports recorded results, not a public replay kit.

Investigation	Evidence class retained privately
Initial and continued Unity work	Pinned source, argument-builder probes, selected upstream results, and environment notes.
Parser reachability	Entrypoint and mocked-launch observations; native parser outcome still absent.
Automatic fallback	Historical lower-layer conflict and automatic fallback behavior, including mocked seams.
MCP boundary and candidate	Original malformed-file tests, valid controls, separate candidate and environment record.
SQL retry	Original shim-based timing probes; upstream cases; both candidate revisions and outcomes.
Five Control cases	Five separate scoped tests, source snapshots, clean-environment checks, selected repairs.
Registered sidecars and lean	Original and candidate case IDs, historical receipts, and separate harness records.

The main portability conditions

Historical parser probes relied on test-only environment assumptions and mocked boundaries; a future replay must rebuild them in a safe isolated setting. The native Claude parser was not executed in the original audit. Reproduction also requires a version-pinned source and controlled dependencies, not merely the examples in this book.

The corrected historical sidecar replay preserved its intended isolated interpreter environment, prevented child-created cache artifacts, and checked package membership before and after. The full native replay requires separately retained original archives; the lean subset is not interchangeable with the missing full evidence.

The private reproduction register retains exact member paths, historical commands, output receipts, versions, and known limitations. It is not part of this public PDF. Publication of findings is not a representation that underlying source code, candidates or complete evidence are available for download.

Evidence C69 | Historical replay instructions, prerequisites and environment records are held privately; not a public kit.

APPENDIX D SOURCE IDENTITIES

Public anchors and private evidence

The findings are tied to historical pinned source identities and recorded local execution receipts. Those receipts, complete audit ZIPs, source-member hashes and candidate files are privately retained by Grounded DI LLC. They are not supplied or independently replayable from this public PDF.

Pinned source and prior-report anchors

Reference	Historical identity or public anchor
Unity Gateway source	Pinned commit b8c80ce4cbce09ea0b11a162136c312917a5853e
SQL Connector source	Pinned commit e7a02cdec620d77502f359138ac1bd693df42c64
Installed distribution	SQL Connector 4.8.0 in the recorded Five Control work
Earlier public report	Databricks SQL Connector GitHub issue #860, opened July 14, 2026

The correction that matters

The Retry-After timing discrepancy had already been described publicly in issue #860 before this audit. Grounded DI reproduced the behavior with its selected local settings and preserved a failed first candidate. It does not claim to have discovered that issue first.

Evidence lineage retained privately

Five unique source packages underlie the original record; a duplicate outer archive and nested copies do not create independent samples. The original engineering ledger preserves exact archive identifiers, member digests, result receipts and claim-to-evidence paths. This public edition intentionally does not enumerate private ZIP names or nested paths.

> PUBLIC EVIDENCE BOUNDARY | Pinned upstream source identifiers and issue #860 are public anchors. The unpublished local execution record is not an independent third-party certification or a public replay bundle.

Public issue: https://github.com/databricks/databricks-sql-python/issues/860

APPENDIX E RELEASE SCOPE

Public claim ceiling and limitations

This public edition presents the recorded behavior of selected local Databricks client methods and candidate repairs. It does not reproduce Grounded DI-specific determinism definitions, private enforcement contracts or internal algorithms. Those omissions do not alter the target findings, case counts, failed attempts or counterevidence.

What a reader may and may not infer

Evidence class	Supported interpretation
Recorded target tests	13 cases: 6 failing assertions in 4 issue groups and 7 passing controls on originals.
Candidate comparison	The same 13 passed separately against local candidates; nothing was deployed or merged.
Unresolved questions	Native Claude parser, hosted outcomes, invalid-input contracts and full replay gaps remain open.
Self-audit record	Grounded DI verifier schema failure, post-test cache and environment corrections are retained.
Source authenticity	Pinned source commits and historical checks, not an independent witness or signed certification.
Release boundary	Public PDF only; audit ZIPs, exact member maps, full candidate code and complete receipts withheld.

Why internal material was withheld

The underlying engineering edition includes exact runtime details, internal control definitions, private paths and implementation-oriented reproduction notes. This release removes those details rather than falsely making the underlying systems publicly reproducible. The narrower local technical results remain intact and challengeable.

> No live Databricks service, production incident, upstream adoption, end-to-end exploit, patent conclusion, product-wide safety grade or third-party certification is established. Independent publication does not imply Databricks sponsorship, approval or endorsement.

Public summary of historical tests; original execution receipts privately retained by Grounded DI LLC.